Unidad 9 - INTERFACES

Concepto de interfaz (Diferencia con Clase Abstracta)

Podríamos crear la clase abstracta Animal, de la que heredarían las clases Gato y Perro, con el método abstracto voz() pero eso descartaría otras clases que no son animales, pero emiten sonido, como Persona o Piano.

En Java disponemos de otra forma de expresar una funcionalidad común de algunas clases que no tienen otra relación entre ellas, en este caso las que tienen implementado el método voz(). A dichas funcionalidades las definiremos en las interfaces. En nuestro ejemplo hablaríamos de la interfaz Sonido, que consiste en implementar el método voz (). Diremos que las clases Perro, Gato y Lobo implementan la interfaz Sonido, ya que, entre sus métodos está voz() , mientras que las clases Caracol y Lagartija no.

Definición de una interfaz

La definición de la interfaz Sonido tendría la siguiente forma:

```
interface Sonido {
     void voz(); //método de la interfaz
}
```

Implementación de una interfaz

Es en la definición de cada clase donde se decide qué hace exactamente voz(). Por ejemplo, la clase Perro se definiría así:

```
class Perro implements Sonido {
     public void voz () {
          System.out.println (" ¡Guau!") ;
     }
```

```
    // Resto de la implementación de Perro
}
```

Como vemos, cuando una clase implementa una interfaz, la declara en la cabecera con la palabra reservada implements seguida del nombre de la interfaz, en nuestro caso, Sonido. Con ello estamos declarando que la clase Perro implementa la interfaz Sonido y que, por tanto, entre sus métodos se encuentra voz(). La implementación de un método de una interfaz en una clase tiene que declararse public, como hemos hecho en nuestro ejemplo. La definición de la clase Gato sería:

```
class Gato implements Sonido {
    public void voz () {
        System.out.println ("¡Miau!");
    }
    //resto de la implementación de Gato
}
```

En cambio, la clase Caracol no implementa la interfaz Sonido y entre sus métodos no encontraremos la implementación de voz().

En definitiva, cuando una clase declara en su encabezamiento que tiene implementada una interfaz concreta, se está comprometiendo a tener implementadas en su definición todas las funcionalidades de esa interfaz. En caso contrario, el compilador se encargará de mostrar un mensaje de error. Las interfaces, sin embargo, no son instanciables por sí mismas; solo podemos crear objetos de las clases que las implementan.

Ejemplos con las interfaces Cola y Pila.

Lo esencial de una cola es que encole y desencole en los extremos opuestos de la estructura de datos utilizada. En el lenguaje de programación, al sitio donde se encola un elemento sele llama final de la cola y al sitio de donde se desencola, cabeza de la cola. De esta forma, el primer elemento que entra en la cola será el primero en salir. En cambio, las funcionalidades de una pila son apilar y desapilar. Ambas operaciones se realizan en el mismo extremo de la pila, llamado cima. Por tanto, en una pila, el último elemento que se apiló será el primero en ser desapilado.

Ejemplo 1

Definir la interfaz Cola para números enteros, e implementarla en la clase Lista, definida en la Actividad resuelta en la unidad anterior.

```
interface Cola{
        void encolar(Integer o);
        Integer desencolar();
}

class Lista implements Cola{
        //…atributos y métodos ya implementados en el capítulo anterior
        void encolar(Integer nuevo){
                insertarFinal(nuevo); //encola al final de la lista
        }
        Integer desencolar(){
                return eliminar(0); //desencola del principio de la lista
        }
}
```

Ejemplo 2

Utilizando la lista anterior, escribir un programa en el que se encolen números enteros introducidos por teclado, hasta que se introduzca uno negativo. A continuación, desencolarlos todos a medida que se muestran por pantalla.

Solución

```
public static void main(String[] args) {
        Lista c = new Lista();
        System.out.print("Introducir número: ");
        Integer n = new Scanner(System.in).nextInt();
        while (n >= 0) {
                c.encolar(n);
                System.out.print("Introducir número: ");
                n = new Scanner(System.in).nextInt();
        }
        n = c.desencolar();
        while (n != null) {       //la cola vacía devuelve null al desencolar
                System.out.print(n+" ");
                n = c.desencolar();
        }
        System.out.println("");
}
```

Ejemplo 3

Definir la interfaz `Pila` para números enteros.

Solución

```
public interface Pila {
        void apilar(Integer elemento); //apila un elemento Integer
        Integer desapilar(); //desapila un elemento y lo devuelve
}
```

Ejemplo 4

Añadir en la clase `Lista` la implementación de la interfaz `Pila`, junto a la interfaz `Cola`, ya implementada.

Solución

```
import java.util.Arrays;
public class Lista implements Pila , Cola{//Se declaran las dos
        .../*resto de implementación de la clase Lista incluidos los de la
            interfaz Cola*/

/*Implementación de Pila: apilamos y desapilamos del final de la lista, que será
la cima (se podría escoger el principio de la tabla igualmente). Recuérdese que
el método eliminar() de la clase Lista devuelve el elemento eliminado */
        public void apilar(Integer elemento) {
                insertarFinal (elemento);
        }
        public Integer desapilar(){ //se extrae el último elemento
                return eliminar(tabla.length - 1);//null si pila vacía
        }
}
```

Atributos de una interfaz

Una interfaz también puede declarar atributos. Por ejemplo, si queremos llevar la cuenta de las sucesivas versiones de nuestra interfaz Sonido, podemos definir el atributo entero version . La definición de la interfaz tendría la siguiente forma:

```
interface Sonido {
     int version = 1;
     void voz();
}
```

El atributo version se convertirá automáticamente en un atributo de

las clases Gato y Perro, y solo podrá cambiarse en la definición de Sonido, pero no en las clases que la implementan, ya que los atributos definidos en una interfaz son static y final por defecto, sin necesidad de escribirlo explícitamente. Se accede a ellos, solo para lectura, a través del nombre de la interfaz o de una clase que la implemente, pero no desde una instancia. Por ejemplo, si queremos mostrar la versión actual de la interfaz Sonido, podemos poner

```
System. out.println (Sonido.version);
```

o bien

```
System.out.println (Perro.version);
```

Una interfaz puede incluso constar solo de atributos. Se pueden usar para añadir un conjunto de constantes compartidas por muchas clases distintas.

Métodos implementados por una interfaz

Hasta ahora solo hemos declarado un método en la interfaz Sonido. Pero, en general, una interfaz puede declarar más de uno. Entre ellos puede haber también métodos implementados en la propia interfaz.

A los métodos declarados pero no implementados, como voz(), se les llama abstractos, igual que en las clases abstractas.

Los que sí están implementados en la interfaz se llaman métodos de extensión y pueden ser tanto estáticos como no estáticos, públicos o privados.

A los métodos de extensión no estáticos públicos se les llama métodos por defecto.

Métodos por defecto

Los métodos por defecto se declaran anteponiendo la palabra reservada default y son public sin necesidad de especificarlo. Vamos

a añadir uno a nuestro ejemplo de los animales. Supongamos que, entre los sonidos de los animales, queremos incluir los ruidos que hacen durmiendo, y que todos emiten el mismo sonido cuando duermen. En este caso podemos incluir, dentro de la interfaz, el método vozDurmiendo(), que va a valer para todos los animales. Por tanto, lo implementamos en la misma interfaz.

```
interface Sonido {
     int version = 1;
     void voz();
     default void vozDurmiendo() {
             System.out.println("Zzz");
     }
}
```

Este método, con su implementación, será incorporado por Perro y Gato automáticamente, y será accesible para todos los objetos de ambas clases.

```
Gato g = new Gato();
g.vozDurmiendo(); //muestra por pantalla <<Zzz>>
Perro p = newPerro();
p.vozDurmiendo(); //también muestra <<Zzz>>
```

No obstante, vozDurmiendo() se puede reimplementar en cada clase haciendo overriding de la implementación que se ha hecho en la interfaz. Supongamos que los leones son una excepción entre los animales, y rugen hasta cuando duermen.

```
class Leon implements Sonido {
     public void voz() {
             System.out.println("¡Grrrr!");
```

```
     }
     @Overriding //de la implementación en Sonido
     public voidvozDurmiendo() {
             System.out.println("¡Grrrr!");
     }
     ... //resto de la implementación de Leon
}
```

En este caso, el código

```
Leon le = new Leon();
le.vozDurmiendo();
```

mostraría por consola «¡Grrrr!».

Recuerda que un método que hace sustitución u overriding de otro implementado en una superclase o una interfaz debe ser declarado public. Al poner default se establece automáticamente como public

Métodos estáticos en una interfaz

En una interfaz también se pueden implementar métodos estáticos. Si no se especifica un modificador de acceso, son public. Estos métodos pertenecen a la interfaz, y no a las clases que la implementan y mucho menos a sus objetos. Por ejemplo, supongamos que todos los animales, sin excepción, emitieran el mismo sonido al bostezar. Entonces podríamos implementar un método estático para los bostezos.

```
interface Sonido {
    //...
    static void bostezo() {//public por defecto
            System.out.println("¡Aaaauuuh!");
    }
}
```

Este método será accesible directamente desde la interfaz Sonido. Para invocarlo tendremos que escribir.

```
Sonido.bostezo();
```

Métodos privados

Además de los métodos abstractos, por defecto y estáticos, que son públicos por defecto, en una interfaz se pueden implementar métodos privados anteponiendo el modificador private. Pueden ser tanto estáticos como no estáticos y no son accesibles fuera del código de la interfaz. Estos métodos están implementados -no son

abstractos- y solo pueden ser invocados por el resto de los métodos no abstractos de la interfaz. En general, son métodos auxiliares para uso interno de otros métodos de la interfaz.

Los métodos declarados, pero no implementados, en una interfaz se dice que son abstractos.

Los métodos no abstractos se implementan dentro de la interfaz y pueden ser estáticos o no estáticos, privados o públicos.

Herencia entre interfaces

Las interfaces pueden heredar unas de otras, heredando todos los atributos y métodos, salvo los privados, y pudiéndose añadir otros nuevos.
A diferencia de las clases, una interfaz puede heredar de más de una interfaz, entre las interfaces es posible la herencia múltiple

Ejemplo:

```
interface Cola{
	void encolar (Integer o);
	Integer desencolar();
}

interface ColaDoble extends Cola{
	void encolarCabeza (Integer nuevo);
	Integer desencolarFinal ();
}
```

A partir de la interfaz Cola, podemos definir la interfaz ColaDoble, que añade dos métodos nuevos, que encolan en la cabeza y desencolan del final de la cola, justo al revés que los métodos originales de Cola. Como se trata de añadir miembros nuevos, hacemos que herede de la interfaz Cola.

Si implementamos la interfaz ColaDoble en la clase Lista, tendremos que implementar los cuatro métodos: los dos definidos en Cola y los dos añadidos en ColaDoble. Por tanto, al implementar ColaDoble, estaremos implementando también Cola de forma automática.

Variables de tipo interfaz

Se pueden crear variables cuyo tipo es una interfaz. Con ellas podremos referenciar cualquier objeto de cualquier clase que la implemente a ella o a una subinterfaz.

```
interface Sonido {
     void voz(); //método de la interfaz
}
```

```
class Perro implements Sonido {
      public void voz () {
            System.out.println (" ¡Guau!") ;
      }
      // Resto de la implementación de Perro
}

class Gato implements Sonido {
      public void voz () {
            System.out.println ("¡Miau!");
      }
      //resto de la implementación de Gato
}
```

Por ejemplo, podremos crear la variable de tipo Sonido.

```
Sonido son;
```

Con ella es posible referenciar objetos de cualquier clase que implemente Sonido.

```
son = new Gato ();
```

La sentencia

```
son.voz();
```

mostrará en la pantalla el mensaje «¡Miau!», ya que se ejecuta el método voz implementado en la clase Gato. Esa misma variable puede referenciar a un objeto Perro.

son = new Perro ();

con lo cual la misma sentencia

son. voz ();

escribirá «¡Guau!», que es el mensaje implementado en la clase Perro.

Este es otro ejemplo de selección dinámica de métodos. La misma línea de código produce efectos distintos ejecuta métodos distintos, dependiendo del objeto referenciado por la variable son. Se decide en tiempo de ejecución qué implementación concreta del método se va a ejecutar.

Es importante señalar que cuando usemos una variable de tipo Sonido para referenciar un objeto de la clase Perro, solo tendremos acceso a los métodos que pertenezcan a la interfaz Sonido y a los heredados de Object, pero no a los otros métodos implementados en la clase Perro. De esta forma, el objeto se manifestará exclusivamente como un objeto que emite un sonido. Esto va a ocurrir con todas las variables de tipo interfaz.

Ejemplo:

Si creamos una lista y la referenciamos con una variable de tipo Cola, veremos que, con esa variable, solo podemos acceder a los métodos encolar() y desencolar() (aparte de los heredados de la clase Object, que siempre son accesibles para cualquier objeto de cualquier clase) y no al resto de los miembros de la clase Lista. De ahí el nombre de interfaz: a través de ella, «vemos» la lista como una cola «pura».

```
interface Cola{
	void encolar (Integer o);
	Integer desencolar();
}
```

```
class Lista implements Cola{

	//atributos y metodos ya implementados en la actividad del tema anterior

	void encolar (Integer nuevo) {
		insertarFinal (nuevo); //encola al final de la lista
	}
	Integer desencolar () {
		return eliminar (0); //desencola del principio de la listaç
	}
}

Cola c = new Lista():
c.encolar(5) ;
System.out.println(c.desencolar()) ;//muestra un 5
```

Al escribir el punto después de la c, NetBeans despliega un menú con los métodos accesibles, entre los que solo figuran los de Cola y los heredados de Object.

Clases anónimas

Lo más común es que las interfaces sean implementadas por clases, de las que luego
se crearán diversos objetos. Sin embargo, hay ocasiones en que una determinada implementación de una interfaz es necesaria en un solo lugar. Cuando ocurre eso, no merece la pena definir una clase nueva para crear un solo objeto del que solo interesan las funcionalidades de la interfaz.
En estos casos, podemos crear sobre la marcha, en el mismo lugar del código donde se
va a usar, un objeto de una clase sin nombre, es decir, anónima, en la que se ñ implementan los métodos de la interfaz que nos interesa. Como no dispondremos de nombre para la clase, tampoco lo tendremos para el constructor. Por eso se usa uno con el nombre de la interfaz. Asimismo, el objeto será referenciado por una variable del tipo de la interfaz. Como ejemplo, crearemos una clase anónima que implemente la interfaz Sonido. Imaginemos que alguien ha

encontrado un animal de una especie desconocida, que emite un extraño ruido. Mientras se identifica o no, para describir su sonido crearemos un objeto de una clase anónima que implementa la interfaz Sonido.

```
Sonido son = new Sonido () {
      public voidvoz () {
            System.out.println("¡Jajeiijojuuuu!");
       }
};
//hasta aquí hemos escrito una sentencia, terminada en punto y
coma.
son.voz();
```

Por pantalla obtendremos el mensaje «¡Jajejijojuuuu!». En realidad, no hemos definido ninguna clase. Lo que hemos hecho es crear un objeto sin clase, que nos permite describir el sonido que emite, aunque es costumbre hablar de clases anónimas. Además, todo esto se implementa en una línea de código ejecutable, no en una clase ni archivo aparte. Por eso se dice que su definición es local.

Recuerda, una clase anónima es la implementación local (en una línea de código) de una interfaz, creándose un objeto único.

Ejemplo 5

Implementar un programa en el que, usando una `Cola` anónima, se encolan números enteros hasta que se introduce un valor negativo. Luego se desencolan todos los valores mostrándolos por pantalla.

Solución

*/*En este ejemplo, en vez de implementar la cola en una lista, construimos una lista dentro de la clase anónima y la usamos para guardar en ella los elementos insertados*/*

```
    Cola cola = new Cola() {
        Lista l = new Lista(); //aquí guardamos los números
        @Override
        public void encolar(Integer nuevo) {
                l.insertarFinal(nuevo);
         }
        @Override
        public Integer desencolar() {
                return l.eliminar(0);
        }
};
    System.out.print("Introducir número(negativo para salir): ");
    Integer n = new Scanner(System.in).nextInt();
    while (n >= 0) {
        cola.encolar(n);
        System.out.print("Introducir número: ");
        n = new Scanner(System.in).nextInt();
    }
    n = cola.desencolar();
    while (n != null) {
        System.out.print(n + " ");
        n = cola.desencolar();
    }
```

Acceso entre los miembros de una interfaz

Es importante destacar que, dentro de una interfaz, todos los métodos default y private tienen acceso entre ellos y, a su vez, pueden acceder a los métodos abstractos. No olvidemos que estos últimos son invocados a través de un objeto de alguna clase (aunque esta sea anónima), donde ya están implementados.

Ejemplo 6

Implementar, en la interfaz `Cola`, el **método** `encolarMultiple()`, que encole un mismo elemento varias veces seguidas.

Solución

*/*Será un método por defecto, que se limitará a llamar varias veces al método abstracto `encolar()`. La interfaz quedará, */*

```
public interface Cola {
        void encolar(Integer nuevo); //método abstracto
        Integer desencolar();      //ídem
        default void encolarMultiple(Integer nuevo,int repeticiones){
                for (int i = 0; i < repeticiones; i++) {
                        encolar(nuevo); //a implementar en la clase
                }
        }
}
```

Sintaxis general

La sintaxis general de la definición de una interfaz tiene la siguiente forma:

```
tipoDeAcceso interface NombreInterfaz {
    //atributos: son public, static y final por defecto:
    tipo atributo1 = valor1;
    //…resto atributos

    //métodos abstractos, sin implementar:
    tipo metodo1(listaParámetros1);
    //…resto métodos abstractos

    //métodos static, implementados:
    static tipo metodoEstatico1(listaParámetros1) {
            //…cuerpo de metodoEstatico1
```

```
    }
    //…resto métodos estáticos

    //métodos por defecto, implementados:
    default tipo metodoDefault1(listaParámetros1) {
            //…cuerpo de metodoDefault 1
    }
    //…resto métodos por defecto

    //métodos privados, solo accesibles dentro de la interfaz
    private tipo metodoPrivado1(listaParámetros1) {
            //…cuerpo de metodoPrivado1
    }
    //…resto métodos privados
}
```

Si se omite tipoDeAcceso, el acceso a la interfaz está restringido al paquete en el que está incluida. Si es public, la interfaz podrá ser importada desde otro paquete mediante una sentencia import.

Una clase puede implementar más de una interfaz, para lo cual deberá declararlas, se paradas por comas, en el encabezamiento, e implementar todos los métodos abstractos de cada una de ellas. En general, la declaración de una clase que implementa una o más interfaces tiene la siguiente forma:

```
tipoDeAcceso class NombreClase implements Interfaz1, Interfaz2,...{
    ...
}
```

Cuando una clase implementa varias interfaces, puede haber algún método declarado en más de una de ellas con el mismo nombre y la misma lista de parámetros (el tipo devuelto tendría que ser también el mismo automáticamente; si no, daría error de compilación)
En este caso, en la clase se hará una única implementación del método.

En general, decimos que una clase implementa una interfaz si tiene implementados to dos sus métodos abstractos, como hemos hecho en nuestros ejemplos. Sin embargo, se puede dejar alguno sin implementar con tal de que la clase se declare como abstracta.

Cuando una interfaz hereda de otra u otras, su definición tendrá la siguiente forma:

```
interface nombreInterfaz extends superInterfaz1, superInterfaz2,…{
     //…definición de la interfaz
}
```